Connect Four (Four-in-a-row) Python Game

Description: Connect Four (or four-in-a-row) is a popular two-player strategy game played on a vertical grid of 6 rows and 7 columns. Players take turns dropping coloured discs into one of the columns, and each disc falls to the lowest available space. The goal is to be the first to form a horizontal, vertical or diagonal line of four of one's own discs. The game combines tactical placement with blocking the opponent's potential connections. This assignment modifies the classic game in the following ways:

1. **Grid size:** Your implementation must allow players to choose the grid size at the start of the game. The grid must have at least 6 rows and 7 columns; smaller grids must not be allowed.
2. **Blocked cells:** At the start of each game, a small number of randomly selected cells (at least one, but fewer than 10% of all cells) are blocked and cannot be used. Discs pass through blocked cells and land in the lowest available unblocked cell of the column (see Figure 1), so a blocked cell always stays empty and cannot form part of a winning line. Your implementation must display blocked cells clearly and handle them correctly when placing discs and when checking for a win or a draw.

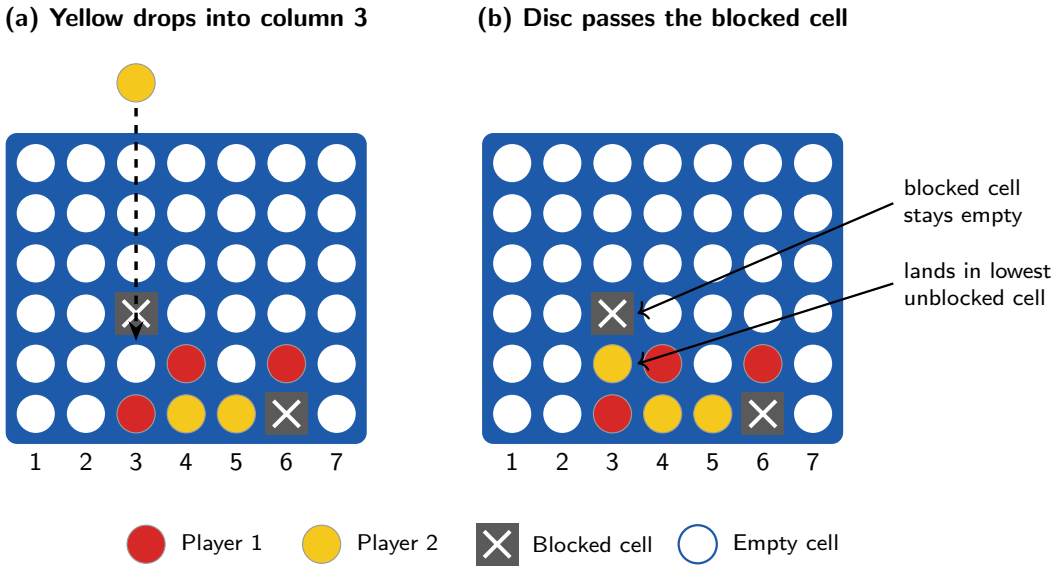


Figure 1: Discs fall through blocked cells and land in the lowest unblocked cell.

Assignment and Submission: Your task is to implement Connect Four game in Python language which satisfies implementation requirements described in the 'Implementation Requirements' part below. To submit your work, you must:

- Create a **private** GitHub repository and provide access to the following usernames: **pastankaitis**,
- Complete and submit **AI coversheet** with your GitHub repository link provided (you link should indicate GitHub *commit hash* with final commit completed before submission deadline).
- **Project demonstration** is a compulsory and assessed part of your assignment and need to take place up to 1 week after the submission. The demonstration will take place in-person and will take around 20 minutes (10 minutes code explanation + 10 minutes for questions).

Implementation Requirements: Implementation of your Connect Four game in Python must meet the following basic requirements:

- You must provide working Python implementation of the game with a text-based user interface and functionality for the user-input (e.g., column number for dropping a token).
- You must demonstrate the process of your game development by creating a **private** Git repository and **performing regular repository updates** (your submission must provide us with the access to the repository).
- Your Git repository must also provide a README.md file which briefly describes your code base and how to execute your code.
- Your implementation must provide some correctness checking. For example, demonstrating that your code correctly implements board game rules via program testing, assert statements or exceptions handling.
- Your implementation must be able to store and export game data.
- Your implementation must support *Human vs Computer* games. For a basic implementation, your AI agent can make random moves or follow a few simple rules.
- Your code must follow good programming practice, for example by organising it into separate Python files (e.g. game logic, user interface, AI agent), using meaningful names and docstrings, and avoiding hard-coded values and repeated code.
- You are required to describe your code and any algorithms you implement by using code comments and/or using Jupyter markdowns.

Suggested advanced features:

(More advanced) AI agent:

- AI agents can be based on variety of heuristic, search-based or machine learning approaches and have different difficulty levels (e.g., easy, medium, hard).
- AI agent's algorithm(s) must be clearly described in the documentation (ReadMe file).

More advanced Graphical User Interface:

- Implement a graphical display of the Connect Four game, providing an interactive visual representation of the board, tokens, and game state.
- Enable full player interaction through the GUI, including the ability for users to select a column to drop a token using mouse clicks or on-screen controls rather than text-based input.

Python Code Optimisation:

- Implement one or more Python optimisation techniques covered in the module.
- Demonstrate the effectiveness of your optimisation by measuring and comparing computation time and memory usage before and after applying the techniques.
- Provide a brief description of your optimisation approach and results in your repository documentation (e.g., README.md or optimisation.md).

AI Usage

You should read the University guidance on the use of generative Artificial Intelligence (AI), for example ChatGPT, at the University. Allowed use of AI assistance in this assignment is **Level 4**: You may use AI extensively throughout your work either as you wish, or as specifically directed in your assessment. Focus on directing AI to achieve your goals while demonstrating your critical thinking.

Marking Scheme

Grade	Requirement
Clear Fail (0–39)	You will fail if the code does not run or fails to meet the minimum described requirements. Failure may also result from submitting code without comments or a written explanation of its functionality. In addition, not granting access to the private Git repository or not demonstrating implementation progress through regular repository updates may also lead to failure. You did not demonstrate your project or were only able to describe the most basic elements of your code.
Marginal Fail (39–49)	You will receive a marginal fail if your code shows partial progress but does not fully meet the minimum requirements. The program may run but contain major errors or missing functionality. Comments or explanations may be limited or unclear, and evidence of consistent repository updates or development progress may be insufficient. You did not demonstrate sufficient understanding of your code. For example, you were unable to explain how key functions work.
Pass (50–59)	To pass, you must submit sufficient working code to show that you have met the requirements of the task, even if not everything works completely. You must include some justifications for your choice of methods, but without mentioning alternatives. Your code demonstration showed only a high-level understanding of your code, and some questions were not answered well.
Merit (60–69)	For a merit, your code must be mostly correct, with only small problems or parts missing, and your comments must be useful rather than simply re-stating the code in English. You showed a very good understanding of your code, although some of your answers could have been clearer, for example by referring to specific parts of your code when explaining design choices.
Distinction (70–100)	For a distinction, your code must be working, correct, and well commented and shows an appreciation of style, efficiency and reliability. All choices for methods and structures are concisely justified, and alternatives are discussed. For a distinction you must go beyond providing a correctly working text-based game implementation with some potential directions provided above. During the code demonstration, you demonstrated a deep understanding of your code and discuss alternative approaches.